

Q: def policy gradient: for notional simplicity, let a denote the

$\Rightarrow$ expectation = the avg value of a variable for all outcomes
 $\Rightarrow$ to maximize expected reward for the policy π :
 (weighted by prob) $\rightarrow$ policy $\rightarrow$ LLM model

$$E[R] = \int p(s) \pi(a|s) R(s, a)$$

$\Rightarrow$ do gradients over the parameters - θ only dependent on policy $\pi(a|s)$

$$\nabla E[R] = \int p(s) \nabla \pi(a|s) R(s, a)$$

not relate to θ ∇ not relate to model params - θ

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T \pi(a_t|s_t) p(s_{t+1}|s_t, a_t)$$

$$\nabla E[R] = \int p(s) \nabla \pi(a|s) \nabla \log \pi(a|s) R(s, a)$$

log-derivative - score function
 trick replacement

$$\nabla E[R] = E[\nabla \log \pi(a|s) R(s, a)]$$

$\Rightarrow$ Naive policy gradient:

①: sample prompt s , sample response $a \sim \pi(a|s)$

②: update parameters based on $\nabla \log \pi(a|s) R(s, a)$: same as SFT, but weighted

$\Rightarrow$ setting: $R(s, a) \in \{0, 1\}$ = whether response is correct or not (by $R(s, a)$)

①: naive policy gradient only updates on correct responses

②: like SFT, but dataset changing over time as policy changes

Challenges: high noise/variance

$\Rightarrow$ sparse rewards (few responses get reward 1, most get 0)

$\Rightarrow$ in contrast: in RLHF, Reward models (learned from pairwise preferences) are more continuous

solution (#1): Baselines: add baseline subtraction to reduce noise / variance in rewards

Recall: $\nabla E[R] = E[\nabla \log \pi(a|s) R(s, a)]$

$\nabla \log \pi(a|s) R(s, a)$ is an unbiased estimate of $\nabla E[R]$,
but maybe there are others with lower variance...

idea: $\Rightarrow$ maximize the baselined reward: $E[R - b(s)]$

$\Rightarrow E[R]$ is shifted by a constant $E[b(s)]$ that doesn't depend on policy π .

$\Rightarrow$ we update based on $\nabla \log \pi(a|s) (R(s, a) - b(s))$

$\Rightarrow$ what $b(s)$ should we use?

two states example: variances reduced from 5.323 to 1.155

1. uniform distribution over (s, a) and $|\nabla \pi(a|s)| = 1$

2. naive_variance = $\text{torch.sed}(\text{torch.tensor}([1, 1, 1, 1, 1, 1, 1, 1, 1, 1]), 9, 0, 2)$

3. define baseline $b(s_1) = 0, b(s_2) = 1$

4. baseline_variance = $\text{torch.sed}(\text{torch.tensor}([1, 1, 1, 1, 1, 1, 1, 1, 1, 1]), 10, 9, 0, 1, 2)$

5. variance: 5.323 $\rightarrow$ 1.155

$\Rightarrow$ optimal $b^*(s) = E[(\nabla \pi(a|s))^2 R | s] / E[(\nabla \pi(a|s))^2 | s]$

still difficult to compute...

so: heuristic way is: mean reward: $b(s) = E[R | s]$

can be estimated